

Experiment No: 2

Date of Performance: 16-02-26

Date of Submission: 03-03-26

Student Name: Shreyans Tatiya

Roll No.: 16010123325

Batch: 3 Div: C

Staff In-charge: Sheetal Pereira

TITLE: : Application of RSA Algorithm for various security services like confidentiality, authentication, signature, non-repudiation and integrity.

Objective: To execute OpenSSL commands and to develop an application.

Expected OUTCOME of Experiment: Student will be able to

CO3: Apply Cryptographic Hash functions system security.

CO5: Illustrate Secure software design principles and apply them for secure software development.

Books/ Journals/ Websites referred:

1. Cryptography and Network Security, Behrouz.A. Fourouzan, Debdeep Mukhopadhyay, McGraw Hill Eighth Edition
2. Cryptography and Network Security: Principles and Practice William Stallings, Pearson Eighth Edition
3. <https://www.openssl.org/>
4. <https://sandilands.info/sgordon/simple-introduction-to-using-openssl-on-command-line>
5. <https://www.cryptool.org/en/cto/openssl/>
6. <https://slproweb.com/products/Win32OpenSSL.html>
7. <https://en.wikipedia.org/wiki/OpenSSL>

Background Theory: (Answer in your handwriting)

SSL, or Secure Sockets Layer, is an encryption-based Internet security protocol. It was first developed by Netscape in 1995 for the purpose of ensuring privacy, authentication, and data integrity in Internet communications. SSL is the predecessor to the modern TLS encryption used today.

A website that implements SSL/TLS has "HTTPS" in its URL instead of "HTTP." Originally, data on the Web was transmitted in plaintext that anyone could read if they intercepted the message. For example, if a consumer visited a shopping website, placed

an order, and entered their credit card number on the website, that credit card number would travel across the Internet unconcealed.

SSL was created to correct this problem and protect user privacy. By encrypting any data that goes between a user and a web server, SSL ensures that anyone who intercepts the data can only see a scrambled mess of characters. The consumer's credit card number is now safe, only visible to the shopping website where they entered it.

SSL also stops certain kinds of cyberattacks: It authenticates web servers, which is important because attackers will often try to set up fake websites to trick users and steal data. It also prevents attackers from tampering with data in transit, like a tamper-proof seal on a medicine container.

OpenSSL is a software library for applications that secure communications over computer networks against eavesdropping or need to identify the party at the other end. It is widely used by Internet servers, including the majority of HTTPS websites.

OpenSSL contains an open-source implementation of the SSL and TLS protocols. The core library, written in the C programming language, implements basic cryptographic functions and provides various utility functions. Wrappers allowing the use of the OpenSSL library in a variety of computer languages are available.

The OpenSSL Software Foundation (OSF) represents the OpenSSL project in most legal capacities including contributor license agreements, managing donations, and so on. OpenSSL Software Services (OSS) also represents the OpenSSL project, for Support Contracts.

OpenSSL is available for most Unix-like operating systems (including Linux, macOS, and BSD) and Microsoft Windows.

Implementation Details (screenshots are to be pasted):

Activity List: You can install OpenSSL on Windows— <https://www.openssl.org/>

OpenSSL at command line: OpenSSL is a program and library that supports many different cryptographic operations, including:

- v Public/private key pair generation**
- v Public key encryption**
- v Hash functions**
- v Certificate creation**
- v Digital signatures**

1. Enlist all the Steps followed and various options explored (with screenshots of your work and keys etc.)

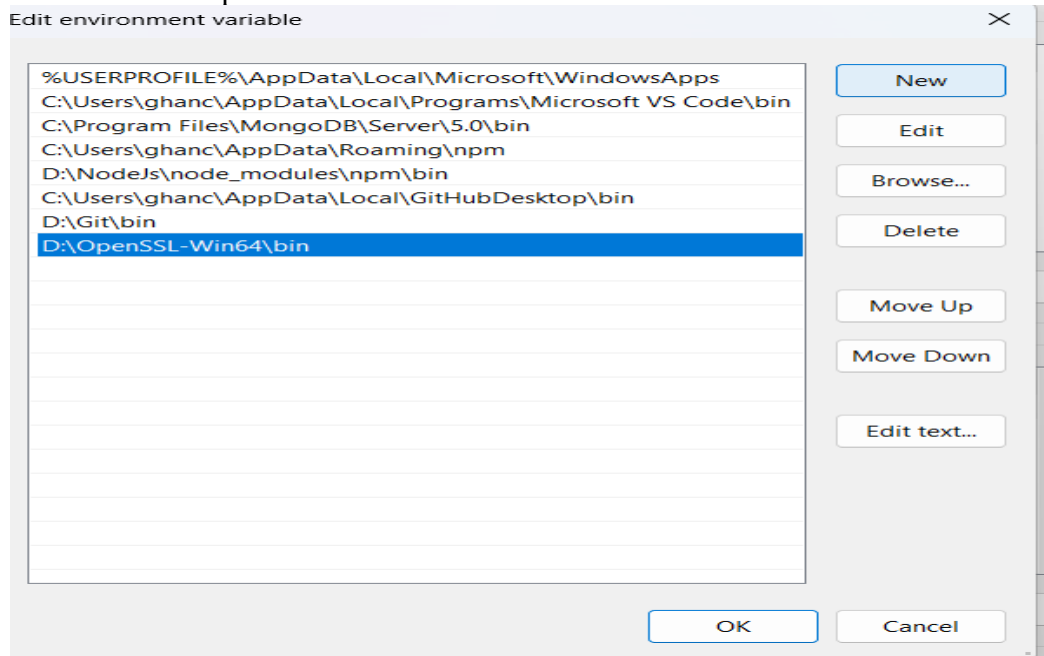
I. Download and Install OpenSSL

1. Download the Download Win64 OpenSSL v3.0.8 (EXE) from the website.

Download Win32/Win64 OpenSSL today using the links below!

File	Type	Description
Win64 OpenSSL v3.0.8 Light EXE MSI	5MB Installer	Installs the most commonly used essentials of Win64 OpenSSL v3.0.8 (Recommended for users by the creators of OpenSSL). Only installs on 64-bit versions of Windows and targets Intel x64 chipsets. Note that this is a default build of OpenSSL and is subject to local and state laws. More information can be found in the legal agreement of the installation.

2. Install and update the Path variable in Environment Variables.

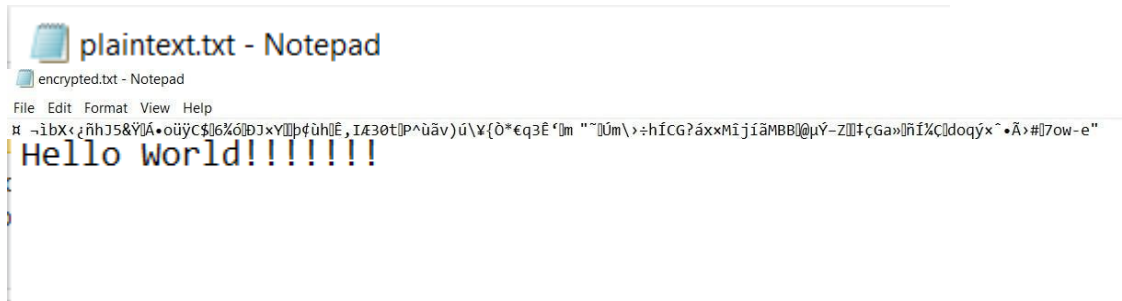


II. Generating RSA private /public key pair

1. Generate key pair using: `openssl genrsa -out mykey1.key 1024`
2. Extract public key from key pair using: `openssl rsa -in mykey1.key -pubout -out mypublickey.key`

III. Public Key Encryption

Using the public key we can encrypt the text in plaintext.txt using: openssl pkeyutl -encrypt -in plaintext.txt -inkey mypublickey.key -pubin -out encrypted.txt



IV. Hash Functions

Create a file to be hashed.

Use the command: openssl dgst -sha256 hashfile.txt



```
C:\Users\Mody\Desktop\IS>openssl dgst -sha256 hashfile.txt
SHA2-256(hashfile.txt)= f1595f567b89de0b48d06b77e107349c79235b1578511860f96253e732f409ea
```

V. Certificate Creation

1. Create a private key using:
openssl genrsa -des3 -out domain.key 2048 Enter
a password

2. Create a certificate using the key using:
openssl req -key domain.key -new -out domain.csr Enter
all the necessary information for the certificate
3. Create a self signed certificate used:
openssl x509 -signkey domain.key -in domain.csr -req -days 365 -out domain.crt
4. View the certificate using:
openssl x509 -text -noout -in domain.crt

VI. Digital signature

1. Create private and public key using:
-openssl genrsa -aes128 -passout pass<phrase>: -out private.pem 4096
-openssl rsa -in private.pem -passin pass:<phrase> -pubout -out public.pem
2. Create a text file

 text.txt - Notepad

File Edit Format View Help

text file

3. Generate the signature of a file using:
-openssl dgst -sha256 -sign<private key> -out /tmp/sign.sha256<file>
-openssl base64 -in /tmp/sign.sha256 -out <signature>

```
C:\Users\Mody\Desktop\IS>openssl dgst -sha256 -sign private.pem -out sign.sha256 text.txt
Enter pass phrase for private.pem:
C:\Users\Mody\Desktop\IS>openssl base64 -in sign.sha256 -out sign.bin
```

4. Verify the signature using:
-openssl base64 -d -in <signature> -out /tmp/sign.sha256

-openssl dgst -sha256 -verify <pub-key> -signature /tmp/sign.sha256
<file>

```
C:\Users\Mody\Desktop\IS>openssl base64 -d -in sign.bin -out sign.sha256  
C:\Users\Mody\Desktop\IS>openssl dgst -sha256 -verify public.pem -signature sign.sha256 text.txt  
Verified OK
```

3. Explain the Importance of the approach followed by you

1. **Public/private key pair generation:** This is essential for creating secure communication channels, as it enables two parties to exchange information in a way that cannot be intercepted or tampered with by a third party.
2. **Public key encryption:** This is another important cryptographic function that allows for secure communication. In this method, data is encrypted with a public key, which can be freely distributed, but can only be decrypted with the corresponding private key.
3. **Hash functions:** These are used to generate a fixed-size string of data from any input data, which is useful for verifying the integrity of information and detecting any changes or tampering.
4. **Certificate creation:** Certificates are used to establish the identity of a person, organization or device, and to authenticate them in secure communications. OpenSSL can be used to create digital certificates for this purpose.
5. **Digital signatures:** These are used to verify the authenticity and integrity of digital messages or documents. OpenSSL can be used to create and verify digital signatures

All of these cryptographic functions are essential for ensuring secure communications and transactions in today's digital world. They provide the foundation for many important applications, including online banking, e-commerce, and secure communication protocols such as SSL/TLS. By using OpenSSL, developers and security professionals can ensure that their systems are properly secured against attacks and breaches.

I.

Determine the current versions of Ubuntu, Linux and OpenSSL

```
root@Shrey:/mnt/c/WINDOWS/system32# openssl version
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
root@Shrey:/mnt/c/WINDOWS/system32# uname -a
Linux Shrey 5.15.153.1-microsoft-standard-WSL2 #1 SMP Fri Mar 29 23:14:13 UTC 2024 x86_64 x86_64 x86_64 GNU/Linux
root@Shrey:/mnt/c/WINDOWS/system32#
```

List Cipher Algorithms in SSL

```
shrey@Shreyas-MacBook-Air-2 ~ %
shrey@Shreyas-MacBook-Air-2 ~ % openssl list-cipher-algorithms
AES-128-CBC
AES-128-CFB
AES-128-CFB1
AES-128-CFB8
AES-128-CTR
AES-128-ECB
AES-128-OFB
AES-128-XTS
AES-192-CBC
AES-192-CFB
AES-192-CFB1
AES-192-CFB8
AES-192-CTR
AES-192-ECB
AES-192-OFB
AES-256-CBC
AES-256-CFB
AES-256-CFB1
AES-256-CFB8
AES-256-CTR
AES-256-ECB
AES-256-OFB
AES-256-XTS
AES128 => AES-128-CBC
AES192 => AES-192-CBC
AES256 => AES-256-CBC
BF => BF-CBC
BF-CBC
BF-CFB
BF-ECB
BF-OFB
CAMELLIA-128-CBC
CAMELLIA-128-CFB
CAMELLIA-128-CFB1
CAMELLIA-128-CFB8
CAMELLIA-128-ECB
CAMELLIA-128-OFB
CAMELLIA-192-CBC
CAMELLIA-192-CFB
CAMELLIA-192-CFB1
CAMELLIA-192-CFB8
CAMELLIA-192-ECB
CAMELLIA-192-OFB
CAMELLIA-256-CBC
CAMELLIA-256-CFB
CAMELLIA-256-CFB1
CAMELLIA-256-CFB8
CAMELLIA-256-ECB
CAMELLIA-256-OFB
CAMELLIA128 => CAMELLIA-128-CBC
CAMELLIA192 => CAMELLIA-192-CBC
CAMELLIA256 => CAMELLIA-256-CBC
CAST => CAST5-CBC
CAST-cbc => CAST5-CBC
CAST5-CBC
CAST5-CFB
CAST5-ECB
CAST5-OFB
ChaCha
DES => DES-CBC
DES-CBC
```

Files Creation

```
/Users/shreya/Documents
shrey@Shreyas-MacBook-Air-2 Documents % echo "This is a sample plaintext file for RSA experiment" > plaintext1.in

shrey@Shreyas-MacBook-Air-2 Documents %
shrey@Shreyas-MacBook-Air-2 Documents %
shrey@Shreyas-MacBook-Air-2 Documents % cp $(which openssl) plaintext2.in
shrey@Shreyas-MacBook-Air-2 Documents % cp /etc/services plaintext3.in

shrey@Shreyas-MacBook-Air-2 Documents % ls -l plaintext*
-rw-r--r--  1 shreya  staff    51  9 Feb 14:27 plaintext1.in
-rwxr-xr-x  1 shreya  staff 1113760  9 Feb 14:29 plaintext2.in
-rw-r--r--  1 shreya  staff  678260  9 Feb 14:29 plaintext3.in
shrey@Shreyas-MacBook-Air-2 Documents %
```

II. Generating RSA private /public key pair

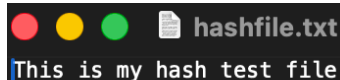
```
shreya@Shreyas-MacBook-Air-2 ~ % cd /Users/shreya/Documents
shreya@Shreyas-MacBook-Air-2 Documents % openssl genrsa -out mykey1.key 1024
Generating RSA private key, 1024 bit long modulus
.....++++
.....++++
e is 65537 (0x10001)
```

```
shreya@Shreyas-MacBook-Air-2 Documents % openssl rsa -in mykey1.key -pubout -out mypublickey.key
writing RSA key
shreya@Shreyas-MacBook-Air-2 Documents %
```

III. Public Key Encryption

```
shreya@Shreyas-MacBook-Air-2 Documents %
shreya@Shreyas-MacBook-Air-2 Documents % openssl rsautl -encrypt \
-in plaintext.txt \
-pubin \
-inkey mypublickey.key \
-out encrypted.txt
Error Reading Input File
8344629376:error:02FFF002:system library:func(4095):No such file or directory:/AppleInternal/Library/BuildRoots/4-CG4rugD0hu_MMyf0tGzXkKREnKN0qX1CS-H2E2k/Library/Caches/com.apple.xbs/Sources/libressl/libressl-3.3/cryptolib/bss_file.c:122:fdopen('plaintext.txt', 'rb')
8344629376:error:20FFF080:BIO routines:CRYPTO_internal:no such file:/AppleInternal/Library/BuildRoots/4-CG4rugD0hu_MMyf0tGzXkKREnKN0qX1CS-H2E2k/Library/Caches/com.apple.xbs/Sources/libressl/libressl-3.3/cryptolib/bss_file.c:125:
shreya@Shreyas-MacBook-Air-2 Documents %
shreya@Shreyas-MacBook-Air-2 Documents %
shreya@Shreyas-MacBook-Air-2 Documents % ls
cipher.txt1.bin      dgst3.bin           domain.csr          My Tableau Repository  mypublickey.key      plaintext1.out       plaintext2.out       plaintext4.in
cipher.txt2.bin      domain.crt          domain.key          mykey1.key            plaintext1.in        plaintext2.in        plaintext3.in
shreya@Shreyas-MacBook-Air-2 Documents %
shreya@Shreyas-MacBook-Air-2 Documents %
shreya@Shreyas-MacBook-Air-2 Documents %
shreya@Shreyas-MacBook-Air-2 Documents %
shreya@Shreyas-MacBook-Air-2 Documents % openssl rsautl -encrypt \
-in plaintext1.in \
-pubin \
-inkey mypublickey.key \
-out encrypted.bin
shreya@Shreyas-MacBook-Air-2 Documents % openssl rsautl -decrypt \
-in encrypted.bin \
-inkey mykey1.key \
-out decrypted.out
shreya@Shreyas-MacBook-Air-2 Documents % cat decrypted.out
This is a sample plaintext file for RSA experiment
shreya@Shreyas-MacBook-Air-2 Documents % cat plaintext1.in
This is a sample plaintext file for RSA experiment
shreya@Shreyas-MacBook-Air-2 Documents %
```

IV. Hash Functions



```
shreya@Shreyas-MacBook-Air-2 Documents % echo "This is my hash test file" > hashfile.txt
shreya@Shreyas-MacBook-Air-2 Documents % openssl dgst -sha256 hashfile.txt
SHA256(hashfile.txt)= ead326068951e48ad7ea3fad9de9ffa9cac5965e5ee3dd05987ce7451cd57c65
shreya@Shreyas-MacBook-Air-2 Documents %
```


V. Certificate Creation

```
shreya@Shreyas-MacBook-Air-2: Documents % openssl genrsa -des3 -out domain.key 2048
Generating RSA private key, 2048 bit long modulus
.....+++++
e is 65537 (0x10001)
[Enter pass phrase for domain.key:
Verifying - Enter pass phrase for domain.key:
shreya@Shreyas-MacBook-Air-2: Documents % openssl req -key domain.key -new -out domain.csr
[Enter pass phrase for domain.key:
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) []: IN
State or Province Name (full name) []: Maharashtra
Locality Name (eg, city) []: Mumbai
Organization Name (eg, company) []: KJSSE
Organizational Unit Name (eg, section) []: Computer Engg
Common Name (eg, fully qualified host name) []: Shreya Menon
Email Address []: shreyam1005@gmail.com

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []: haining1005
shreya@Shreyas-MacBook-Air-2: Documents % openssl x509 -signkey domain.key -in domain.csr -req -days 365 -out domain.crt
Signature ok
subject=C=IN, ST=Maharashtra, L=Mumbai, O=KJSSE, OU=Computer Engg, CN=Shreya Menon/emailAddress=shreyam1005@gmail.com
Getting Private key
[Enter pass phrase for domain.key:
```

```
shreya@Shreyas-MacBook-Air-2: Documents % openssl x509 -text -noout -in domain.crt
Certificate:
Data:
  Version: 1 (0x0)
  Serial Number:
    f0:25:8e:9a:04:ec:84:e7
  Signature Algorithm: sha256WithRSAEncryption
  Issuer: C=IN, ST=Maharashtra, L=Mumbai, O=KJSSE, OU=Computer Engg, CN=Shreya Menon/emailAddress=shreyam1005@gmail.com
  Validity
    Not Before: Feb  9 09:45:16 2026 GMT
    Not After : Feb  9 09:45:16 2027 GMT
  Subject: C=IN, ST=Maharashtra, L=Mumbai, O=KJSSE, OU=Computer Engg, CN=Shreya Menon/emailAddress=shreyam1005@gmail.com
  Subject Public Key Info:
    Public Key Algorithm: rsaEncryption
    RSA Public-Key: (2048 bit)
    Modulus:
      00:ca:2d:9f:dc:ff:5a:6a:ce:d2:7d:60:dc:2b:df:
      b7:bc:65:c9:75:87:51:6a:3e:d7:1f:77:33:e9:7d:
      25:a4:3b:98:a3:75:09:01:25:b9:fc:92:e1:b2:2f:
      f5:25:89:e4:ef:e4:9e:06:a4:f2:dd:42:b6:ef:a0:
      e3:34:27:7b:5d:47:7c:52:dc:e9:67:40:fe:8a:de:
      0c:00:a3:0c:d2:43:96:4e:bf:d1:b6:8d:62:35:d8:
      ef:e6:10:fc:9d:c3:31:58:52:7d:66:56:81:23:2b:
      9a:37:66:ef:0e:a7:49:62:84:fa:fe:e4:b3:9f:23:
      49:0b:c2:4c:cf:8e:19:da:00:2f:1e:90:cd:0a:fd:
      03:32:ca:e6:9d:42:75:b6:ca:04:d2:5a:5b:36:e8:
      d2:7c:ea:3b:23:fa:26:45:79:22:97:77:74:27:73:
      b2:d9:8b:d4:ee:6f:28:94:d4:55:30:fe:67:27:3e:
      5e:9c:50:8b:fd:97:ff:de:70:94:40:78:98:51:b8:
      ea:fb:43:55:01:55:49:1a:e5:83:69:c8:55:3f:ab:
      d7:15:b6:c0:79:1c:e3:ad:96:40:6f:66:2b:85:92:
      55:c8:28:2a:8c:5f:d9:cc:90:fa:ad:46:fd:07:74:
      34:21:c0:62:e1:6d:37:b0:bd:86:82:f7:7b:60:ba:
      5e:61
    Exponent: 65537 (0x10001)
  Signature Algorithm: sha256WithRSAEncryption
    2c:82:df:7d:51:3d:97:b7:b9:8f:57:f6:f3:b1:1d:0f:26:16:
    41:fa:3c:39:f9:a1:d2:7d:8c:20:77:8e:51:fa:04:e0:79:40:
    ed:ee:99:8b:da:09:ee:6a:04:a1:08:02:1a:ba:7a:fa:1f:36:
    b2:7b:cd:fc:89:5f:12:6d:51:c6:09:f4:4a:ff:8a:f2:cd:15:
    fe:cd:17:c6:33:db:7f:ad:37:6d:e0:05:b0:e1:ee:bf:59:11:
    32:85:94:74:04:7c:06:94:8e:20:98:95:d7:e0:fa:ce:21:9f:
    9a:7e:04:19:16:7e:e1:95:3b:2b:24:f5:e4:fb:38:9a:b0:
    b4:e7:ee:c2:e5:10:7e:f2:29:41:41:57:9c:ba:9a:3a:79:80:
    51:dc:21:bf:07:1c:1a:c1:67:1c:34:1f:de:39:c5:b9:0f:90:
    b9:94:04:7a:8f:0e:1a:79:5e:77:08:d6:c4:02:7c:ba:3a:3e:
    d4:b0:32:b5:02:4f:d1:dd:02:3e:f1:11:01:30:77:9a:6c:31:
    52:1f:35:f1:04:b6:03:b1:fb:de:da:e8:04:1d:d1:df:0b:e7:
    6d:39:7d:aa:25:ad:78:53:a8:2f:29:d3:0e:81:64:75:48:e3:
    1a:fc:48:81:81:6a:ad:72:9a:0d:d4:1f:6b:08:e1:cc:8f:a9:
    c7:2a:11:b9
shreya@Shreyas-MacBook-Air-2: Documents %
```

VI. Digital signature

```
root@Shrey:~# echo "Miaoo" > message.txt
root@Shrey:~# openssl rsautl -encrypt \
-inkey public.pem -pubin \
-in message.txt -out encrypted.bin
The command rsautl was deprecated in version 3.0. Use 'pkeyutl' instead.
root@Shrey:~# openssl pkeyutl -encrypt \
-pubin -inkey public.pem \
-in message.txt -out encrypted.bin
root@Shrey:~# openssl pkeyutl -decrypt \
-inkey private.pem \
-in encrypted.bin -out decrypted.txt
root@Shrey:~# cat decrypted.txt
Miaoo
root@Shrey:~# openssl dgst -sha256 -sign private.pem \
-out signature.bin message.txt
root@Shrey:~# openssl dgst -sha256 \
-verify public.pem \
-signature signature.bin message.txt
Verified OK
root@Shrey:~# openssl dgst -sha256 message.txt
SHA2-256(message.txt)= ea0f5e66ec92a7eac9a8efa56f0dc067428f95845acd89571a5e9cccbcc90137
root@Shrey:~#
```

```
shreya@Shreyas-MacBook-Air-2 Documents % openssl base64 -in /tmp/sign.sha256 -out signature.txt
shreya@Shreyas-MacBook-Air-2 Documents % cat signature.txt
oY29wQZHTbH3MjYzQ2axK0U6YxWUvTaMLrNc0Iar4jGnC3XeTrJQrL6GHjWFK
gDh+ILLksd6FHgUlo/khzJRRNXkl.v6iIF7S2utk33+LAzISkeq9FAdBAuu/QFG
GzWsnKrcuuNZiHcwz15APEizU3+VQ/uwFZB2oN+JSzeIgT0S6wAgtDrzQTB/Q/
8cNTaIEa2QmTTA+uu7r3PpCsW61tXGoGu19zP9MMQrBje8pn59YCTnxqzqHw/zt
btrR1VaEo1eniEFLgZZ/Zdi+IY2yRj0qfdlnDfzcFYpecJRHkyE1zxs1517xYSLE
obDrLTTwsmw/9j589bNlpuN3S7hZsYa0+1EFxNTj0xSS00BYQ0yXa2D8LKsuef4I
sAsnHRaM8xy1XQ5Izi0E9Kq+U6hY14hUzpaWRdFvabHmQxxqo6qHMQB/waHsIMB
GgxPZ16LlchpZoeowNR0/4rxax01CULRK9+jkrVT0xL2APTD3JJKRVRxQUYf4ctn
Bmo4Wq662QhGSAKGAkXezRYrKAuf4XFZxvTi6JUGBm3TULMeQ0v63mDol3cYJe8
DQr8XU0yZKvacs2VSPxjmbayk+YnTtWb89vaCQlWFFFFJHuKmqFvq4QIoYUyWISwg
0228q7NuLDCMkexFVv26gtm1yGzdAtk/+crcds1QmPQ=
shreya@Shreyas-MacBook-Air-2 Documents %
shreya@Shreyas-MacBook-Air-2 Documents %
shreya@Shreyas-MacBook-Air-2 Documents % openssl base64 -d -in signature.txt -out /tmp/sign.sha256
shreya@Shreyas-MacBook-Air-2 Documents % openssl dgst -sha256 -verify public.pem -signature /tmp/sign.sha256 message.txt
Verified OK
shreya@Shreyas-MacBook-Air-2 Documents %
```

Conclusion:

In this experiment, public key cryptography was performed using OpenSSL. Keys were generated, a file was digitally signed, and the signature was successfully verified, certificate was generated along with performing RSA and use of hash functions.

Post Lab Questions:

- 2.1 In the RSA algorithm, $p=7$, $q=11$ and $e=13$, then what will be the value of d ?
- 2.2 Discuss various cryptanalysis attacks possible to be carried out on RSA
- 2.3 Comment on drawbacks of RSA. Discuss solution(s) over the same.

2.1

$p=7$
 $q=11$
 $e=13$

1. Compute n

$$n = p \times q$$

$$= 7 \times 11$$

$$= 77$$
2. $\phi(n)$

$$\phi(n) = (p-1)(q-1)$$

$$= 6 \times 10$$

$$= 60$$
3. Compute d

$$d = e^{-1} \bmod \phi(n)$$

$$= 13^{-1} \bmod 60$$

$$13d \equiv 1 \bmod 60$$

$d = 37$

2.2

- ① Brute force Attack : If prime no.s are small $\rightarrow$ RSA breaks easily
- ② Factorization Attack: Pollard's rho, Quadratic sieve
- ③ Timing Attack : Attacker measures time taken for decryption to deduce private key.

2.3

<u>Drawbacks</u>	<u>Solutions</u>
1. Slow for large data	1. Use Hybrid Encryption
2. High computation cost	- RSA encrypts symm. key
3. Large key size required	- AES encrypts data
4. Not secure to quantum attacks	2. Proper padding
	3. Use post quantum algo.
	4. Large key sizes.